AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING

M M VASANTH

CH.SC.U4CSE24227

Week – 8

Design and Analysis of Algorithm(23CSE211)

1. JOB SEQUENCING

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Job {
    char id;
    int dead;
    int profit;
};

int compare(const void* a, const void* b) {
    struct Job* temp1 = (struct Job*)a;
    struct Job* temp2 = (struct Job*)b;
    return (temp2->profit - temp1->profit); // Sort descending by
profit
}

int min(int num1, int num2) {
    return (num1 > num2) ? num2 : num1;
}

void printJobScheduling(struct Job arr[], int n) {
    qsort(arr, n, sizeof(struct Job), compare);

    int result[n];
    bool slot[n];

    for (int i = 0; i < n; i++) slot[i] = false;

    for (int i = 0; i < n; i++) {
```

```
        for (int j = min(n, arr[i].dead) - 1; j >= 0; j--) {
            if (slot[j] == false) {
                result[j] = i;
                slot[j] = true;
                break;
            }
        }
    }

    printf("Following is maximum profit sequence of jobs: \n");
    int totalProfit = 0;
    for (int i = 0; i < n; i++) {
        if (slot[i]) {
            printf("%c ", arr[result[i]].id);
            totalProfit += arr[result[i]].profit;
        }
    }
    printf("\nTotal Profit: %d\n", totalProfit);
}

int main() {
    struct Job arr[] = { {'a', 2, 100}, {'b', 1, 19}, {'c', 2, 27},
                         {'d', 1, 25}, {'e', 3, 15} };
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Name: M M VASANTH\n");
    printf("Roll No: CH.SC.U4CSE24227\n\n");
    printJobScheduling(arr, n);
    return 0;
}
```

Output:

```
vasanth@VasanthMM:/mnt/c/Users/vasan$ nano js.c
vasanth@VasanthMM:/mnt/c/Users/vasan$ gcc js.c -o js
vasanth@VasanthMM:/mnt/c/Users/vasan$ ./js
Name: M M VASANTH
Roll No: CH.SC.U4CSE24227

Following is maximum profit sequence of jobs:
c a e
Total Profit: 142
vasanth@VasanthMM:/mnt/c/Users/vasan$
```

⏱ Time Complexity: O(N^2)

Sorting the jobs takes O(N log N). Then, iterating through the jobs and finding a free slot takes O(N^2) in the worst case. This can be optimized to O(N log N) using Disjoint Sets.

📦 Space Complexity: O(N)

Arrays result and slot take linear memory space to track the schedule.

2. HUFFMAN CODING

Code:

```
#include <stdio.h>

#include <stdlib.h>


#define MAX_TREE_HT 100


struct MinHeapNode {

    char data;

    unsigned freq;

    struct MinHeapNode *left, *right;

};


struct MinHeap {

    unsigned size;

    unsigned capacity;

    struct MinHeapNode** array;

};


struct MinHeapNode* newNode(char data, unsigned freq) {

    struct MinHeapNode* temp = (struct
MinHeapNode*)malloc(sizeof(struct MinHeapNode));

    temp->left = temp->right = NULL;

    temp->data = data;

    temp->freq = freq;

    return temp;
```

```
}


struct MinHeap* createMinHeap(unsigned capacity) {

    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct
MinHeap));

    minHeap->size = 0;

    minHeap->capacity = capacity;

    minHeap->array = (struct MinHeapNode**)malloc(minHeap->capacity *
sizeof(struct MinHeapNode*));

    return minHeap;

}


void swapMinHeapNode(struct MinHeapNode** a, struct MinHeapNode** b) {

    struct MinHeapNode* t = *a;

    *a = *b;

    *b = t;

}


void minHeapify(struct MinHeap* minHeap, int idx) {

    int smallest = idx;

    int left = 2 * idx + 1;

    int right = 2 * idx + 2;


    if (left < minHeap->size && minHeap->array[left]->freq < minHeap-
>array[smallest]->freq)

        smallest = left;
```

```
    if (right < minHeap->size && minHeap->array[right]->freq <
minHeap->array[smallest]->freq)



 smallest = right;



    if (smallest != idx) {

        swapMinHeapNode(&minHeap->array[smallest], &minHeap-
>array[idx]);

        minHeapify(minHeap, smallest);

    }

}



int isSizeOne(struct MinHeap* minHeap) {

    return (minHeap->size == 1);

}



struct MinHeapNode* extractMin(struct MinHeap* minHeap) {

    struct MinHeapNode* temp = minHeap->array[0];

    minHeap->array[0] = minHeap->array[minHeap->size - 1];

    --minHeap->size;

    minHeapify(minHeap, 0);

    return temp;

}



void insertMinHeap(struct MinHeap* minHeap, struct MinHeapNode*
minHeapNode) {
```

```
 ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq)
{




  minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = minHeapNode;
}

void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;
    for (int i = (n - 1) / 2; i >= 0; --i)
        minHeapify(minHeap, i);
}

void printArr(int arr[], int n) {
    for (int i = 0; i < n; ++i)
        printf("%d", arr[i]);
    printf("\n");
}

int isLeaf(struct MinHeapNode* root) {
```

```
    return !(root->left) && !(root->right);

}


struct MinHeap* createAndBuildMinHeap(char data[], int freq[], int
size) {

    struct MinHeap* minHeap = createMinHeap(size);

    for (int i = 0; i < size; ++i)

        minHeap->array[i] = newNode(data[i], freq[i]);




    minHeap->size = size;

    buildMinHeap(minHeap);

    return minHeap;

}


struct MinHeapNode* buildHuffmanTree(char data[], int freq[], int
size) {

    struct MinHeapNode *left, *right, *top;

    struct MinHeap* minHeap = createAndBuildMinHeap(data, freq, size);


    while (!isSizeOne(minHeap)) {

        left = extractMin(minHeap);

        right = extractMin(minHeap);

        top = newNode('$', left->freq + right->freq);

        top->left = left;

        top->right = right;
```

```
        insertMinHeap(minHeap, top);
    }
    return extractMin(minHeap);
}

void printCodes(struct MinHeapNode* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {



        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
    if (isLeaf(root)) {
        printf("%c: ", root->data);
        printArr(arr, top);
    }
}

void HuffmanCodes(char data[], int freq[], int size) {
    struct MinHeapNode* root = buildHuffmanTree(data, freq, size);
    int arr[MAX_TREE_HT], top = 0;
```

```
    printCodes(root, arr, top);
}

int main() {
    char arr[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
    int freq[] = { 5, 9, 12, 13, 16, 45 };
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Name: M M VASANTH\n");
    printf("Roll No: CH.SC.U4CSE24227\n\n");
    printf("Huffman Codes generated:\n");
    HuffmanCodes(arr, freq, size);
    return 0;
}
```

Output:

```
vasanth@VasanthMM:/mnt/c/Users/vasan$ nano hc.c
vasanth@VasanthMM:/mnt/c/Users/vasan$ gcc hc.c -o hc
vasanth@VasanthMM:/mnt/c/Users/vasan$ ./hc
Name: M M VASANTH
Roll No: CH.SC.U4CSE24227

Huffman Codes generated:
f: 0
c: 100
d: 101
a: 1100
b: 1101
e: 111
vasanth@VasanthMM:/mnt/c/Users/vasan$
```

⏱ Time Complexity: O(N log N)

Extracting the minimum from the heap and inserting the new combined node back into the heap takes O(log N) time. Since we do this for N characters, the total time is N log N.

📦 Space Complexity: O(N)

The Min-Heap array and the Huffman Tree require memory proportional to the number of distinct characters N.